1 Лекция 16. Почему FPT-задача может не иметь полиномиального ядра

1.1 Почему быстрый решатель и сильная предобработка — разные возможности

Ядеризация за время, полиномиальное по длине исходного входа (x, k), выдаёт эквивалентный вход (x', k') той же задачи с размером не больше $g(k)$. Полиномиальное ядро требует $g(k) = (k+1)^{O(1)}$. В FPT-алгоритме множитель $f(k)$ может быть произвольной вычислимой функцией. Поэтому FPT-решатель нельзя просто запустить до конца и назвать полиномиальной предобработкой.

В этой главе примером станет **k-Path** — существование простого пути на k вершинах. Color coding и его дерандомизация дают FPT, но при стандартном сложностном предположении полиномиального ядра нет. Мы объясним способ доказательства: объединить много трудных входов так, чтобы положительный ответ означал положительность хотя бы одного из них, а параметр оставался маленьким.

Это операция логического **OR**, то есть «хотя бы один». Операция AND означала бы «все», и подмена здесь меняет теорию. Важно также различать размер всего объединённого входа и его параметр. Первому разрешено расти с количеством объединяемых экземпляров, второму — только очень ограниченно.

1.1.1 Что означает сложностное предположение

Класс coNP/poly допускает полиномиальную по длине входа строку совета (advice), общую для **всех** входов этой длины. Сама строка не обязана вычисляться. Получив её, машина работает в ко-недетерминированном полиномиальном времени: на YES-входе все ветви принимают, на NO-входе хотя бы одна отвергает. Эквивалентно, отрицательные ответы имеют NP-подобные сертификаты при помощи общего совета.

Предположение $\mathrm{NP} \not\subseteq \mathrm{coNP/poly}$ означает, что такого способа не хватает для всех NP-задач. Оно влечёт $P \neq NP$: при $P = NP$ всякая NP-задача решалась бы детерминированно полиномиально и тем более лежала бы в coNP/poly. Обратное следствие не утверждается.

Ниже результат о невозможности сжатия списков связывает это предположение с ядрами. Это отдельная математическая теорема, а не аргумент «информации слишком много». Полное доказательство этой теоремы включено после определения дистилляции; затем из неё выводится конкретная нижняя оценка.

1.2 Между произвольным и полиномиальным ядром

В лекции 3 доказано: всякая FPT-задача имеет некоторое ядро. Его размер может быть экспоненциальным или хуже. Поэтому поиск полиномиального ядра — дополнительный вопрос, не эквивалентный FPT.

Полиномиальная компрессия (polynomial compression) разрешает выдавать эквивалентную строку другой задачи, а не обязательно экземпляр той же самой. Время остаётся полиномиальным, размер — $k^{O(1)}$. В частности, каждое полиномиальное ядро есть полиномиальная компрессия.

OR-distillation языка L в язык R получает любое число строк $x_1, \ldots, x_r$, за время, полиномиальное по сумме длин, выдаёт y длины $\mathrm{poly}(\max_i |x_i|)$, причём $y \in R$ тогда и только тогда, когда хотя бы одно $x_i \in L$. Размер результата не зависит от числа строк.

Теорема о дистилляции (Fortnow–Santhanam, в книге теорема 15.3 и следствие

15.4): если NP-трудный язык допускает такую OR-distillation, то NP $\subseteq$ coNP/poly. Класс coNP/poly допускает для всех входов одной длины общую полиномиальную advice-строку. Это стандартное предположение о невозможности такого включения сильнее формулировки «P не равно NP». Докажем этот факт, после чего выпишем перенос к конкретной задаче.

1.3 Доказательство теоремы о дистилляции

Сначала докажем более сильную формулировку: если язык L допускает OR-дистилляцию A в любой язык R, то $L \in$ coNP/poly. Предполагаем двоичный алфавит; постоянный другой алфавит перекодируется без существенного изменения длин.

Зафиксируем длину n. Пусть $K = p(n)$ — полиномиальная верхняя граница длины результата дистилляции на строках длины до n, а $t = K + 1$. Рассмотрим множество S всех NO-строк языка L длины до n. Выход A на любой t-ке элементов S не принадлежит R.

Скажем, что строка результата y **покрывает** вход x, если существует t-ка строк длины до n, содержащая x, на которой A выдаёт y. Если $y \notin R$, она не может покрыть YES-вход: присутствие хотя бы одного YES в списке заставляет результат лежать в R.

Покажем, что все строки S можно покрыть не более чем $n + 1$ результатами, не принадлежащими R. Пусть S' — ещё не покрытые строки. Возможных выходов длины до K меньше $2^{K+1} = 2^t$. Поэтому некоторый результат $y \notin R$ достигается более чем на $|S'|^t/2^t = (|S'|/2)^t$ кортежах из $(S')^t$.

Пусть $C \subseteq S'$ — все строки, встречающиеся хотя бы в одном из этих кортежей. Все кортежи лежат в C^t, откуда $|C|^t \geq (|S'|/2)^t$ и $|C| \geq |S'|/2$. Результат y покрывает все C, то есть по крайней мере половину оставшихся строк. Добавляем y в список советов и повторяем. Первоначально $|S| < 2^{n+1}$; после $n+1$ уменьшений вдвое непокрытых строк не остаётся. Если S пусто, список пуст с самого начала.

Полученный список Y из не более $n+1$ строк длины до K имеет полиномиальный размер и подходит одновременно для всех NO-входов длины n. Его используем как advice. Не требуется уметь вычислять Y: определение неравномерного класса требует его существования.

Теперь дополнение L имеет NP-проверку с этим советом. Для входа x угадываем t-ку строк длины до n, содержащую x, запускаем A и проверяем, что результат есть в Y. Длина сертификата не больше tn, а время A полиномиально по ней, значит, проверка полиномиальна по n. Для NO-входа подходящая t-ка есть по свойству покрытия. Для YES-входа ни одна не подойдёт, поскольку её результат лежит в R, а все элементы Y — вне R. Получено $L \in$ coNP/poly.

Если L NP-труден относительно полиномиальных many-one редукций, любой язык L' из NP сводится к L. Выход редукции на входах длины n имеет длину не больше некоторого полинома $q(n)$. В совет для L' можно включить советы для L всех длин до $q(n)$: полиномиальное число строк полиномиальной длины всё ещё имеет полиномиальный общий размер. Запускаем редукцию и проверку для нужной длины. Значит, все NP-языки лежат в coNP/poly, что и требовалось.

1.4 OR-cross-composition

Полиномиальное отношение эквивалентности на строках должно проверяться за полиномиальное время и разбивать строки длины не больше N на полиномиально много классов. Например, графы можно группировать по числу вершин; некорректные кодировки вынести в отдельный класс.

OR-cross-composition из классической задачи L в параметризованную Q получает r входов одного класса и за полиномиальное время по их суммарному размеру выдаёт (y, k) так, что

$$(y,k) \in Q \iff \exists i\ x_i \in L, \qquad k \le \mathrm{poly}(\max_i |x_i| + \log r).$$

Само y может быть большим: ограничен только параметр. **Теорема:** если NP-трудный L OR-cross-composes в Q, то у Q нет полиномиальной компрессии, если NP $\not\subseteq$ coNP/poly.

1.5 Доказательство переноса

Предположим наличие компрессии C. Чтобы дистиллировать произвольный список входов длины максимум N, сначала удалим дубликаты: различных двоичных строк длины не больше N меньше 2^{N+1}, поэтому $\log r = O(N)$. Удаление дубликатов не меняет OR и полиномиально по исходной сумме длин.

Разобьём входы по полиномиальному отношению на не более $\mathrm{poly}(N)$ групп. Для каждой применим cross-composition: параметр $k_j \le \mathrm{poly}(N)$, затем компрессию: строка z_j длины $\mathrm{poly}(N)$. Выдадим список этих строк с однозначными разделителями. Его длина полиномиальна по N, а целевой язык принимает такой список, если хотя бы одна строка принадлежит языку компрессии. Получена OR-distillation NP-трудной задачи. Доказанная теорема о дистилляции даёт запрещённое включение классов.

Обратите внимание: мы не «потеряли информацию по принципу Дирихле», а построили алгоритм дистилляции и применили конкретный сложностной результат. Простого информационного лозунга для доказательства недостаточно.

1.6 Полный пример: k-Path

Исходная классическая задача — Hamiltonian Path в простом неориентированном графе; она NP-полна. Группируем графы по одинаковому числу n вершин. По входам $G_1, \ldots, G_r$ строим их непересекающееся объединение H и ставим параметр $k = n$.

Простой путь на n вершинах целиком лежит в одной компоненте H. Каждая копия G_i имеет ровно n вершин. Следовательно, такой путь существует в H тогда и только тогда, когда один из G_i имеет гамильтонов путь. Параметр $k = n$ ограничен длиной одного входа, а время и размер H полиномиальны по сумме. Некорректные кодировки дают фиксированный NO; пустые/тривиальные графы можно обработать непосредственно. Все условия cross-composition выполнены.

Итак, **k-Path не имеет полиномиальной компрессии и, в частности, полиномиального ядра, если** NP $\not\subseteq$ coNP/poly. При этом лекции 9–10 дают FPT-алгоритмы. Это явный пример различия двух вопросов.

1.7 Перенос через PPT

Polynomial parameter transformation (PPT) из A в B работает за полиномиальное время, сохраняет ответ и даёт $k' \le \mathrm{poly}(k)$. Если B имеет полиномиальную компрессию, композиция с PPT даёт такую же для A. Следовательно, отсутствие компрессии переносится от A к B. Обычная FPT-редукция для этого недостаточна: она может тратить $2^k N^c$ времени или выдавать $k' = 2^k$.

Для переноса именно отсутствия **ядер** без усиления до компрессии могут понадобиться дополнительные условия и обратная классическая редукция. Здесь используем сильный вывод об отсутствии компрессии, чтобы не скрывать этот нюанс.

1.8 Выходной контроль

Для новой задачи задайте параметр; найдите ветвление/ядро/DP или редукцию; сформулируйте точное время, модель и условную гипотезу. Вы должны различать NP-трудность, W[1]-трудность, ETH-запрет субэкспоненты и отсутствие полиномиального ядра: они отвечают на разные вопросы. Освоение курса проверяется написанными вами доказательствами, а не числом просмотренных видео.

1.9 Практика

Условия задач 16.1–16.5. Решения открывайте после своей попытки.

1.10 Источники и дополнительное чтение

Письменный источник: Cygan et al., *Parameterized Algorithms*, авторская копия 30.05.2016: §15.1, с. 524–533; §15.2.2, с. 537–538. PDF. Номера страниц печатные; номер страницы PDF больше на 16.

Изложение, поясняющие примеры и упражнения подготовлены для этого курса. Ссылки фиксируют происхождение результатов; для чтения конспекта видео не требуется. Необязательные видео собраны в программе.